ESTUDO DE CASO

Infraestrutura de Dados Financeira

Detecção de fraudes e anomalias em tempo real

Disciplina: Análise de Dados | Mineração de Dados | Aprendizado de Máquina

Professor: Benir Falcão Pistille

Integrantes: Eduardo Kamo, Gustavo Lopes, Iago Rossan, Lucas Vinicius

Data: 02 de setembro de 2026

Introdução

O estudo de caso apresenta uma empresa financeira de grande porte, com atuação nacional e internacional, diversas unidades de negócio, funcionários e parceiros digitais. Nesse ambiente, transações, acessos e operações de investimento são gerados continuamente. Parte desses eventos pode representar fraude ou algum comportamento fora do padrão, por isso a análise precisa acontecer com rapidez e sem comprometer o registro histórico das informações.

A solução proposta organiza o fluxo em cinco camadas: ingestão, streaming, armazenamento, processamento e consumo. O Apache Kafka recebe os eventos; o Apache Flink faz a análise em tempo real; o MinIO guarda os dados e os modelos; o Apache Spark executa os processamentos em lote e o retreinamento; e o PostgreSQL disponibiliza os resultados para relatórios, consultas e dashboards. O Apache Airflow atua de forma transversal, coordenando as tarefas e acompanhando o funcionamento dos pipelines.

No caso analisado, a plataforma deve detectar uma possível fraude em menos de 100 milissegundos, operar com alta disponibilidade e ainda manter dados suficientes para auditoria, investigação e melhoria do modelo. Assim, a arquitetura atende tanto à necessidade de resposta imediata quanto à análise histórica usada pelas áreas de negócio (PISTILLE, 2026).

Visão geral da arquitetura

Camada e ferramenta	Responsabilidade	Informações do caso
01 - Ingestão Apache Kafka	Receber e distribuir os eventos em tempo real.	Transações, logs de acesso, cotações, contas, canal, IP, localização e horário.
02 - Streaming Apache Flink	Aplicar regras e o modelo de fraude com baixa latência.	Score de fraude, regra acionada, anomalia, decisão e latência.
03 - Armazenamento MinIO	Manter o data lake com dados brutos e processados.	Histórico de transações, logs, dados de treino, resultados e versões dos modelos.
04 - Processamento Apache Spark	Executar ETL, análises históricas e retreinamento.	Atributos calculados, perfil comportamental, indicadores e modelo atualizado.
05 - Consumo PostgreSQL	Organizar os resultados para BI, relatórios e APIs.	Fraudes por dia, indicadores por canal, falsos positivos e volume de transações.
Orquestração Apache Airflow	Agendar, coordenar e monitorar os pipelines.	Dependências, tentativas, falhas, duração dos jobs e alertas técnicos.

1. Associação entre ferramentas, camadas e funções

a) Qual ferramenta recebe os dados em tempo real? O que aconteceria se ela ficasse fora do ar?

O Apache Kafka é responsável pela entrada dos dados em tempo real. Ele recebe as mensagens produzidas pelos aplicativos, pelo internet banking, pelos caixas eletrônicos, pelas plataformas de investimento e pelos demais sistemas ligados à empresa. Depois, disponibiliza esses eventos para os consumidores, principalmente o Flink. Como trabalha com tópicos e partições, o Kafka consegue distribuir uma grande quantidade de mensagens e permite o processamento paralelo (APACHE SOFTWARE FOUNDATION, [s. d.]c).

Se o serviço ficasse completamente indisponível, as novas transações deixariam de chegar ao processo de detecção naquele momento. Isso causaria atraso nos alertas e acúmulo de eventos nos sistemas de origem. Em uma implantação real, esse risco é reduzido com vários servidores, replicação das mensagens, confirmações de gravação e tentativas de reenvio. Portanto, a tolerância a falhas depende da configuração do cluster e não apenas da ferramenta.

b) Qual é a diferença entre o Flink e o Spark? Por que os dois são necessários?

O Flink processa os eventos à medida que eles chegam. No momento da transação, combina regras de negócio com o modelo de aprendizado de máquina, calcula o risco e informa se a operação pode seguir ou se precisa ser bloqueada. Também consegue analisar janelas de tempo, por exemplo, várias transferências realizadas pela mesma conta em poucos minutos (APACHE SOFTWARE FOUNDATION, [s. d.]b).

O Spark trabalha principalmente com o histórico armazenado no MinIO. Em execuções programadas, realiza limpeza e integração dos dados, calcula características do comportamento dos clientes, produz indicadores e retreina o modelo. Os dois são necessários porque o Flink atende à decisão imediata, enquanto o Spark usa o passado para melhorar as decisões futuras. O modelo treinado e validado pelo Spark pode ser versionado no MinIO e depois utilizado pelo Flink (APACHE SOFTWARE FOUNDATION, [s. d.]d).

c) Qual é o papel do MinIO?

O MinIO funciona como o data lake central. Nele ficam os eventos originais, os resultados produzidos pelo Flink, os logs, os conjuntos de treinamento, os resultados dos processamentos em lote e as diferentes versões dos modelos. Manter o dado bruto e o dado processado permite reproduzir uma análise, investigar uma ocorrência e verificar quais informações foram usadas em determinada decisão (MINIO, [s. d.]).

Como o conteúdo é financeiro e pode incluir dados pessoais, o armazenamento precisa ter controle de acesso, criptografia, política de retenção e registro das consultas realizadas. Esses cuidados tornam o repositório útil para análise sem perder a segurança e a rastreabilidade.

d) Por que o Airflow aparece separadamente, gerenciando as outras camadas?

O Airflow é o responsável pela orquestração. Ele define a ordem das tarefas, agenda execuções, controla dependências, registra o andamento dos jobs e informa quando alguma etapa falha. Um fluxo diário pode, por exemplo, verificar a chegada dos dados no MinIO, iniciar um processamento no Spark, validar o resultado e depois atualizar as tabelas do PostgreSQL (APACHE SOFTWARE FOUNDATION, [s. d.]a).

Sua função é diferente da função do Kafka. O Kafka transporta os eventos contínuos da operação; o Airflow coordena tarefas que possuem dependências e horários de execução. Por isso ele aparece como um componente transversal, ligado às várias camadas da plataforma.

2. Análise do fluxo de uma transação suspeita

A situação proposta envolve uma transferência de R$ 47.000 realizada às 3h da manhã, por um dispositivo nunca utilizado pela conta e com destino a outro país. Cada característica, sozinha, pode ter uma justificativa legítima. Entretanto, a combinação entre valor elevado, horário incomum, dispositivo novo e destino internacional aumenta o risco e exige avaliação imediata.

01. Em qual camada a transação entra? Quais variáveis são registradas?

A transação entra pela camada de ingestão, no Apache Kafka. Os campos apresentados no estudo são id_transacao, valor_R$, timestamp, id_conta_origem, id_conta_destino, canal, ip_dispositivo e localizacao_GPS. Nesse exemplo, o valor seria R$ 47.000 e o horário estaria registrado por volta das 3h. A origem e o destino permitem relacionar a movimentação às contas envolvidas, enquanto o canal, o IP e a localização ajudam a caracterizar o contexto do acesso.

Para reconhecer que o dispositivo é novo e que o destino está em outro país, o evento precisa ser comparado com informações cadastrais e com o histórico da conta. Esse enriquecimento pode usar o identificador do aparelho, o país da instituição beneficiária e o perfil comportamental calculado anteriormente.

02. Em qual camada a decisão de bloquear ou aprovar é tomada? Em quanto tempo?

A decisão é tomada no Flink, na camada de streaming. O evento recebido do Kafka passa pelas regras de negócio e pelo modelo de fraude. O processamento gera um score, registra a regra acionada, indica se houve anomalia e produz a decisão de aprovação ou bloqueio. Conforme a meta apresentada no estudo, essa análise deve ocorrer em menos de 100 milissegundos, antes da aprovação definitiva.

No cenário analisado, os quatro sinais de risco provavelmente aumentariam a pontuação. Dependendo da política da instituição, a transação poderia ser bloqueada ou submetida a uma autenticação adicional. A decisão e seus motivos precisam ficar registrados para uma eventual investigação.

03. Onde a transação fica armazenada para análise futura e retreinamento?

A transação é armazenada no MinIO. Além do evento original, convém guardar o resultado da análise, o score, as regras acionadas, a decisão, a latência e a versão do modelo utilizada. Essa trilha torna possível explicar por que a operação foi bloqueada ou liberada.

Após a análise da equipe responsável, a ocorrência pode receber um rótulo confirmado de fraude ou de transação legítima. Esse retorno passa a fazer parte dos dados de treinamento usados pelo Spark, permitindo que o modelo aprenda com os acertos e com os erros anteriores.

04. Ao final do dia, onde o gestor consegue ver quantas fraudes foram bloqueadas?

O gestor consulta os resultados em um dashboard ou relatório alimentado pelo PostgreSQL. O Spark consolida os eventos do período e o pipeline grava informações estruturadas em tabelas como fato_fraudes_dia, kpi_canal e volume_transacoes. Com esses dados, o painel pode apresentar quantidade de bloqueios, valores envolvidos, distribuição por canal, horário e região, além da evolução ao longo do tempo (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026).

A análise não deve se limitar ao número absoluto de ocorrências. Também é importante comparar a quantidade de tentativas com o volume total de cada canal. Um canal muito utilizado pode registrar mais fraudes em números absolutos e, ainda assim, ter uma taxa proporcionalmente menor.

05. Se o modelo bloquear uma transação legítima, qual camada precisa ser ajustada e como?

Esse tipo de erro é chamado de falso positivo. O bloqueio acontece durante a análise do Flink, mas a melhoria do modelo ocorre principalmente no Spark. Primeiro, a equipe confirma que a transação era legítima e registra essa informação no histórico do MinIO. Depois, o Spark utiliza o exemplo corrigido em um novo treinamento e verifica se os atributos, os pesos e o limite de decisão estão adequados.

A nova versão precisa ser comparada com a anterior antes da implantação. O objetivo é reduzir falsos positivos sem aumentar a quantidade de fraudes que passam despercebidas. Depois de aprovada, a versão é armazenada no MinIO e disponibilizada ao Flink. Caso o problema esteja em uma regra fixa ou no limite de bloqueio, essa configuração também deve ser ajustada no Flink. A tabela taxa_falso_positivo, no PostgreSQL, ajuda a acompanhar o resultado da mudança.

3. Objetivos de negócio: conexão entre dados e decisões

1) Qual canal tem mais tentativas de fraude?

Essa pergunta é respondida na camada de consumo. Os dados do PostgreSQL podem ser agrupados por canal para comparar aplicativo, internet banking, caixa eletrônico e outras origens. O dashboard deve apresentar tanto a quantidade de tentativas quanto a taxa em relação ao total de transações. Com essa visão, a empresa pode reforçar autenticação, limites, comunicação preventiva e monitoramento nos canais que apresentam maior risco.

2) Em qual camada será executado o aprendizado do modelo?

O treinamento e o retreinamento são executados no Spark, na camada de processamento, usando os dados históricos e rotulados do MinIO. Depois de validado, o modelo é aplicado pelo Flink às novas transações. Em outras palavras, o Spark aprende com os exemplos anteriores e o Flink utiliza esse aprendizado em tempo real.

3) Em qual camada serão disparados os jobs de coleta e as tarefas?

Os jobs programados são disparados e acompanhados pelo Airflow. Ele pode iniciar coletas periódicas, processamentos do Spark, validações de qualidade, retreinamentos e cargas no PostgreSQL. Os eventos de transação, por outro lado, continuam chegando pelo Kafka. A diferença é que o Kafka trabalha com o fluxo contínuo de mensagens, enquanto o Airflow coordena processos com início, fim e dependências definidas.

4) Onde ocorre a visualização das transações e dos alertas de fraude?

A visualização gerencial ocorre em relatórios e dashboards de BI alimentados pelo PostgreSQL. Neles, gestores e auditores podem acompanhar fraudes por período, canal, região, valor e situação. Já o alerta de uma transação específica é produzido pelo Flink durante a análise em tempo real e pode ser enviado à central antifraude.

Também é necessário separar o alerta de negócio do alerta técnico. O Flink indica o risco de fraude associado à transação. O Airflow informa falhas ou atrasos nos pipelines. Essa distinção ajuda a encaminhar cada ocorrência para a equipe responsável.

4. Diagrama das áreas auditadas e do fluxo da plataforma

O diagrama a seguir reúne as principais origens de dados, os pontos observados pela auditoria, o caminho percorrido pelas informações e as áreas que utilizam os resultados. As setas contínuas representam o fluxo de dados e decisões. As setas tracejadas mostram a atuação do Airflow na coordenação e no monitoramento dos processos.

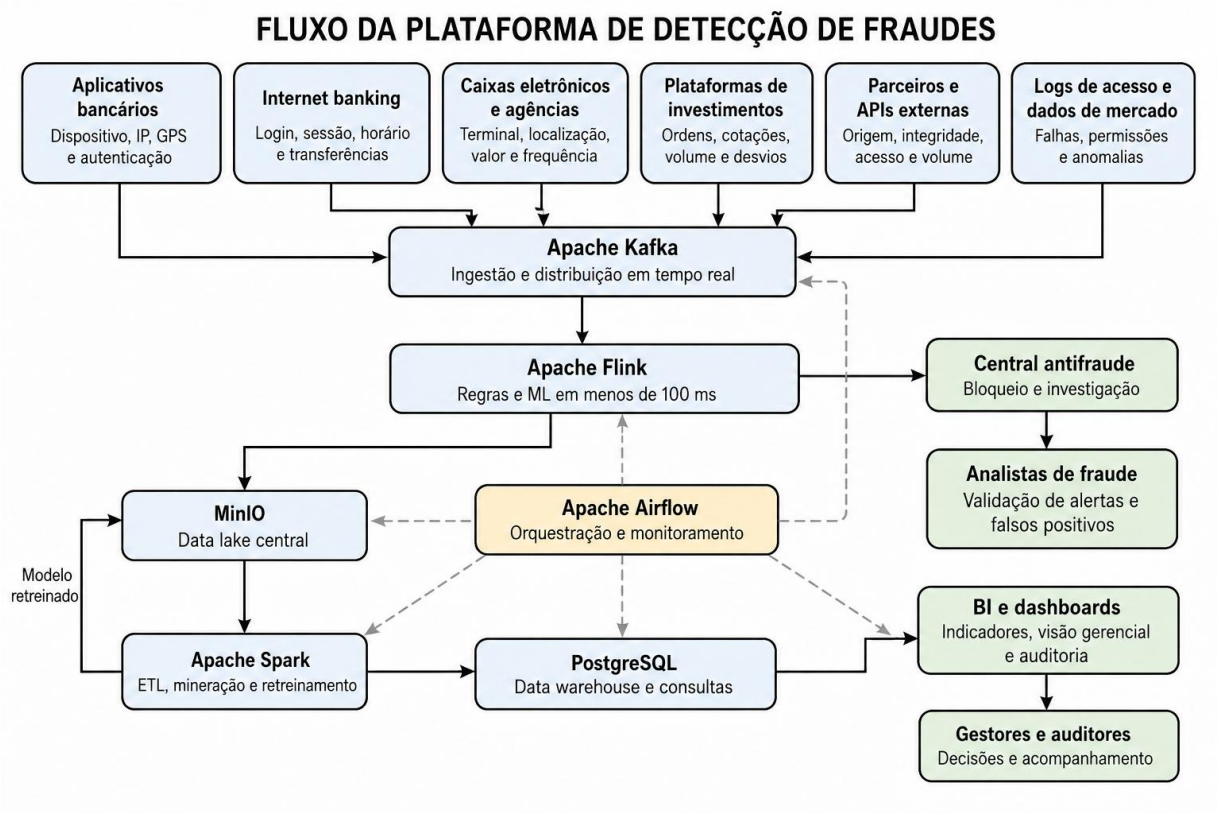


Figura 1 - Fluxo da plataforma de detecção de fraudes e áreas auditadas

Fonte: elaboração própria, com base no estudo de caso (2026).

Os aplicativos bancários, o internet banking, os caixas eletrônicos, as agências, as plataformas de investimento, os parceiros e os registros de acesso enviam eventos ao Kafka. Nesses pontos são observados dados como dispositivo, IP, localização, autenticação, horário, valor, frequência, origem, destino, permissões e comportamento de mercado.

O Flink analisa as transações e encaminha situações de risco à central antifraude. Os eventos e resultados são mantidos no MinIO, utilizados pelo Spark nos processamentos históricos e consolidados no PostgreSQL. Os dashboards atendem gestores e auditores, enquanto os analistas validam alertas e falsos positivos. O retorno das análises permite retreinar o modelo e fechar o ciclo de melhoria contínua.

Conclusão

A arquitetura proposta combina resposta em tempo real com análise histórica. Kafka e Flink formam o caminho mais rápido, responsável por receber e avaliar as transações antes da aprovação. MinIO e Spark preservam e processam o histórico, tornando possível investigar ocorrências e melhorar o modelo. PostgreSQL organiza os resultados para consulta e tomada de decisão, enquanto o Airflow mantém os processos coordenados e monitorados.

No exemplo da transferência de R$ 47.000, a plataforma reúne diferentes sinais de risco, calcula uma pontuação, toma uma decisão em poucos milissegundos e registra os motivos. Se a decisão estiver errada, o caso é revisado e volta ao processo de treinamento. Dessa forma, o sistema não apenas bloqueia operações suspeitas, mas também produz conhecimento para reduzir perdas e aperfeiçoar os controles da instituição financeira.

Referências

APACHE SOFTWARE FOUNDATION. *Apache Airflow documentation: DAGs*. [S. l.], [s. d.]a. Disponível em: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html. Acesso em: 2 set. 2026.

APACHE SOFTWARE FOUNDATION. *Apache Flink: stateful computations over data streams*. [S. l.], [s. d.]b. Disponível em: https://flink.apache.org/. Acesso em: 2 set. 2026.

APACHE SOFTWARE FOUNDATION. *Apache Kafka: introduction*. [S. l.], [s. d.]c. Disponível em: https://kafka.apache.org/intro/. Acesso em: 2 set. 2026.

APACHE SOFTWARE FOUNDATION. *MLlib: main guide*. Apache Spark Documentation. [S. l.], [s. d.]d. Disponível em: https://spark.apache.org/docs/latest/ml-guide.html. Acesso em: 2 set. 2026.

MINIO. *MinIO: high-performance, S3-compatible object storage*. GitHub, [s. d.]. Disponível em: https://github.com/minio/minio. Acesso em: 2 set. 2026.

PISTILLE, Benir Falcão. *Estudo Técnico: Infraestrutura Unificada para Processamento de Dados em Média e Larga Escala*. Versão 2.0. Setembro de 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 18.6 documentation*. [S. l.], 2026. Disponível em: https://www.postgresql.org/docs/current/. Acesso em: 2 set. 2026.